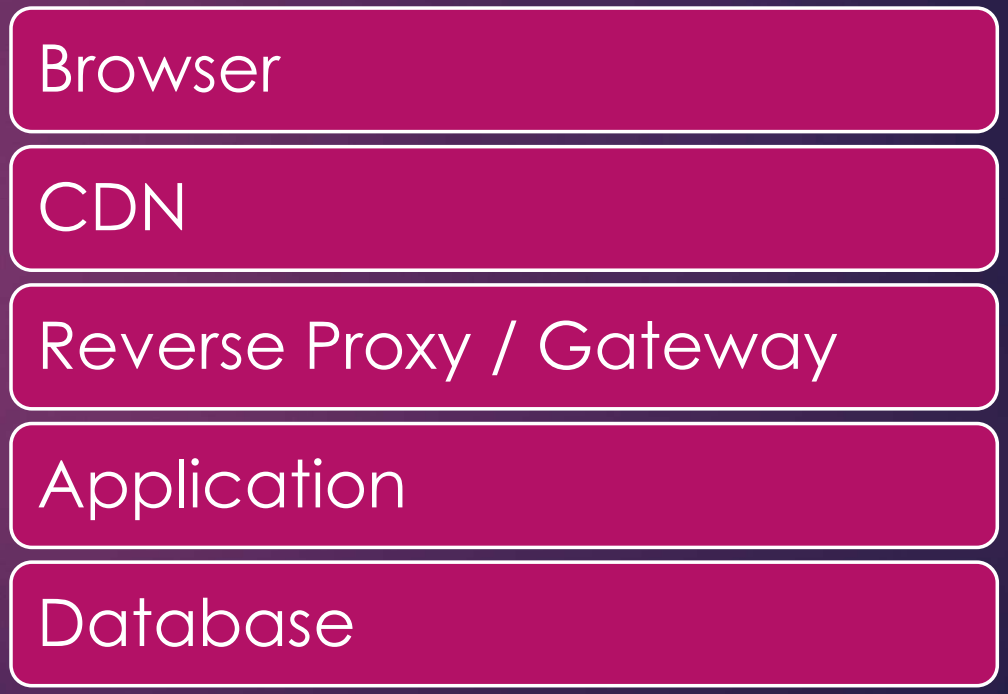

Explore • Learn • Engage • Socialize
JAVA MEETUP
#SKG
10 YEARS
VOXXED DAYS
THESSALONIKI
SKG
Java
Meetup
Caching in Java
CACHE SMARTER
MEMORY CACHE
LOCAL CACHE
DISTRIBUTED CACHE
16.09.2026
19:00
OK!Thess

What caching actually is

- A cache stores the result of expensive work so later reads avoid repeating it
- **Expensive:** DB queries, external APIs, CPU-heavy calculations, file reads, serialization, aggregations
- **Trade-off:** less work / lower latency **vs** freshness / memory / consistency / complexity

Where can caching happen?

- **Browser:** `Cache-Control: max-age=3600`, `ETag`
- **CDN:** static content, public API responses
- **Application:** ← this talk
  `ConcurrentHashMap`, Caffeine, Redis, Hazelcast



The demo application

- `GET /products/{id}`
  → `ProductService.getProduct(id)`
  → repository
- Spring Boot 4.1.1, Java 21
- One commit per stage

Product.java · ProductRepository.java

```java
public record Product(Long id, String name, Double price) {}

@Repository
class ProductRepository {
    public Product findById(Long id) {
        dbSlowOperation();   // Thread.sleep(1000)
        return new Product(id, "iPhone", 1200.0);
    }
}
```

▶ SWITCH TO IDE

Stage 0: No cache

What to observe

- Run the app after the checkout
- Every request takes ~1.0 s
- Can we avoid doing the same work again?

Just use a Map

ProductService.java

```
private final Map<Long, Product> cache = new HashMap<>();

public Product getProduct(Long id) {
   Product cached = cache.get(id);
   if (cached != null) {
      return cached;
   }
   Product product = productRepository.findById(id);
   cache.put(id, product);
   return product;
}
```

Request 1: ~1000 ms | Request 2: ~1 ms | So… are we done?

What is wrong with the HashMap cache?

- Not thread-safe
- No **TTL**, no maximum size, no eviction → memory grows forever
- No **statistics**, no **loading** coordination
- No **invalidation** strategy
- No **distributed** behavior

A slightly better cache

ProductService.java

```
private final Map<Long, Product> cache = new ConcurrentHashMap<>();

public Product getProduct(Long id) {
   return cache.computeIfAbsent(id, productRepository::findById);
}
```

Thread-safe, one line, per-key loading coordination | When does this value disappear from memory?

▶ **SWITCH TO IDE**

Stages 1–2: HashMap → ConcurrentHashMap

What to observe

- Second call is instant
- Loop: memory grows, nothing is ever removed
- Enhancement: Atomic operation

Caffeine

ProductService.java

```
Cache<Long, Product> cache = Caffeine.newBuilder()
      .maximumSize(10)
      .expireAfterWrite(Duration.ofMinutes(1))
      // .expireAfterAccess(Duration.ofMinutes(1))
      .build();

public Product getProduct(Long id) {
   return cache.get(id, productRepository::findById);
}
```

com.github.ben-manes.caffeine:caffeine, version managed by Boot | A Map with policies around the data

Caffeine policies and statistics

- **maximumSize** bounds memory
- **expireAfterWrite** / **expireAfterAccess**
- **recordStats** → hit rate, loads, evictions
- **evictionListener** shows entries leaving
- `GET /products/cache/stats`
  → `CacheStats{hitCount, missCount, evictionCount…}`

Cache initialization

```
Cache<Long, Product> cache = Caffeine.newBuilder()
      .maximumSize(10)
      .expireAfterWrite(Duration.ofMinutes(1))
      .evictionListener((key, value, cause) -> {
         System.out.println("Evicting product: " + key);
      })
      .recordStats()
      .build();
```

LoadingCache

- The cache now knows how to obtain a missing value
- **refreshAfterWrite:** serve the old value, reload in the background
- **expireAfterWrite:** after 1 minute the entry really expires
- The service method is one line

Loading Cache initialization

```
final LoadingCache<Long, Product> cache =
Caffeine.newBuilder()
       .maximumSize(10)
       .refreshAfterWrite(Duration.ofSeconds(10))
       .expireAfterWrite(Duration.ofMinutes(1))
       .recordStats()
       .build(productRepository::findById);

public Product getProduct(Long id) {
   return cache.get(id);
}
```

▶ **SWITCH TO IDE**

Stages 3-5: Caffeine

What to observe

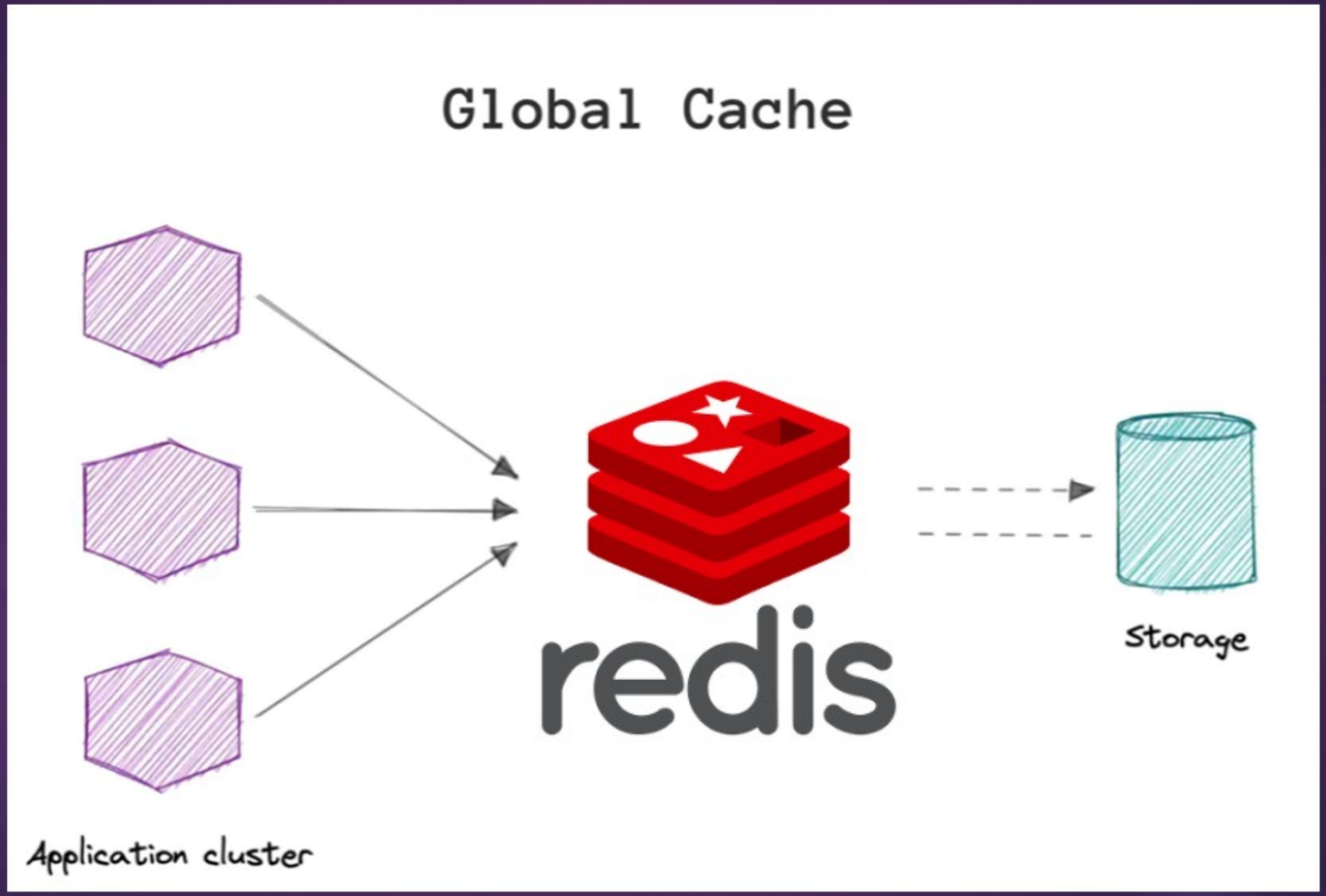
- Stats after a hit and a miss
- Ids 1..11 exceed `maximumSize(10)` → `Evicting product: …` in the console
- LoadingCache: `cache.get(id)` only, reload happens in the background
- Transition: what happens on 10 pods?

Local cache topology

```
Instance A          Instance B
---------           ---------
Caffeine            Caffeine
{1 -> Product}      {}
```

- Same request on instance B → **MISS**
- Every JVM has its own view of the world

Shared Cache - Redis



Shared cache with Redis

Advantages

- Shared values across instances
- Survives application restarts
- Centralized TTL

Costs

- Network call
- Serialization
- Infrastructure
- Availability
- Operational complexity

Redis, by hand

ProductService.java

```
public Product getProduct(Long id) {
   String key = "product:" + id;

   Product cached = redisTemplate.opsForValue().get(key);
   if (cached != null) {
      return cached;
   }

   Product product = productRepository.findById(id);
   redisTemplate.opsForValue().set(key, product, Duration.ofMinutes(10));
   return product;
}
```

spring-boot-starter-data-redis · RedisTemplate<String, Product> · Do we want this in every business method?

Redis needs a serializer

- Default is Java serialization
  `Needs Serializable payload`
- String keys → readable in `redis-cli`
- JSON values
  → `{"id":1,"name":"iPhone 16", "price":1200.0}`
- Boot 4 / Jackson 3:
  `JacksonJsonRedisSerializer`

RedisConfig.java

```
@Bean
RedisTemplate<String, Product> productRedisTemplate(
      RedisConnectionFactory cf) {

   var template = new RedisTemplate<>();
   template.setConnectionFactory(cf);
   template.setKeySerializer(new StringRedisSerializer());
   template.setValueSerializer(
         new JacksonJsonRedisSerializer<>(Product.class));
   return template;
}
```

▶ **SWITCH TO IDE**

Stages 6-7: Redis

What to observe

- curl: 1 s, then 0 s
- MONITOR: 1st request `GET` + `SET … PX 600000`, 2nd only `GET`
- Restart the **app**, curl again → still a hit
- UI: Redis Commander `:8081`, Redis Insight `:5540`

Do we really want this boilerplate?

IF CACHED … LOAD … PUT … EVICT … IN EVERY BUSINESS METHOD

@Cacheable is not a cache

```
@Cacheable
      ↓
Spring Cache abstraction
      ↓
CacheManager
      ↓
ConcurrentMap / Caffeine / Redis / JCache …
```

- `@Cacheable` describes caching behavior
- The `CacheManager` decides where the values actually live

Minimal Spring Cache example

App.java · ProductService.java

```
@EnableCaching
@SpringBootApplication
public class App { ... }

@Service
public class ProductService {
   @Cacheable("products")
   public Product getProduct(Long id) {
      return productRepository.findById(id);
   }
}
```

spring-boot-starter-cache (Boot 4: the auto-configuration lives in spring-boot-cache)

What happens behind @Cacheable

- Spring wraps the bean in a **proxy**
- The method body executes **only on a cache miss**
- Key = method arguments by default (1)
- Cache name = `products`

conceptually, what the proxy does

```
public Product getProduct(Long id) {
    Product cached = cache.get(id);
    if (cached != null) {
        return cached;
    }
    Product result = repository.findById(id);
    cache.put(id, result);
    return result;
}
```

Same annotation, three providers

application.yaml

```
spring.cache.type: simple

spring.cache.type: caffeine
spring.cache.caffeine.spec:
maximumSize=10,expireAfterWrite=1m,recordStats

spring.cache.type: redis
spring.cache.redis.time-to-live: 1m
```

Startup log

```
CacheManager: ConcurrentMapCacheManager

CacheManager: CaffeineCacheManager

CacheManager: RedisCacheManager
```

The Java method does not change!!

Boot 4 gotchas

CacheConfig.java

```
@Bean
RedisCacheConfiguration redisCacheConfiguration(CacheProperties cacheProperties) {
   return RedisCacheConfiguration.defaultCacheConfig()
         .entryTtl(cacheProperties.getRedis().getTimeToLive())
         .serializeValuesWith(SerializationPair.fromSerializer(
               new JacksonJsonRedisSerializer<>(Product.class)));
}
```

- `spring.cache.type` must be explicit: auto-detection puts **Redis before Caffeine**
- `RedisCacheManager` defaults to JDK serialization → `Serializable` or a `RedisCacheConfiguration` bean
- A `RedisCacheConfiguration` bean **ignores** `spring.cache.redis.*` → inject `CacheProperties` for the TTL
- Jackson 3: `tools.jackson.*`, `JacksonJsonRedisSerializer` (not `Jackson2…`)

▶ **SWITCH TO IDE**

Stages 8-10: providers

What to observe

- After each restart: the `CacheManager:` startup line
- Changes are YAML only, the third adds `CacheConfig`
- Redis key is `products::1` (cache name, double colon, key) vs `product:1` before

Cache keys

SpEL keys

```
@Cacheable(cacheNames = "products", key = "#id")
public Product getProduct(Long id) { ... }

@Cacheable(cacheNames = "search", key = "#category + ':' + #page")
public List<Product> search(String category, int page) { ... }
```

Updating data: the stale-data bug

ProductController.java

```java
@PutMapping("/{id}")
public Product update(@PathVariable Long id,
                      @RequestParam Double newPrice) {
    return productService.update(id, newPrice);
}
```

Database = new value, cache = old value. No annotation on update.

What the client sees

```
GET  /products/1                → 1200.0
PUT  /products/1?newPrice=999   → 999.0
GET  /products/1                → 1200.0   ← stale
```

@CacheEvict and @CachePut

ProductRepository.java

```
@CacheEvict(cacheNames = "products", key = "#id")
public Product update(Long id, Double newPrice) { ... }

@CachePut(cacheNames = "products", key = "#result.id")
public Product update(Long id, Double newPrice) { ... }
```

Three annotations

```
@Cacheable  → may skip the method on a hit

@CachePut   → always runs the method, stores the result

@CacheEvict → removes the entry
```

▶ **SWITCH TO IDE**

Stages 11-13: invalidation

What to observe

- 3rd call **still 1200.0**; show `products::1` in Redis Insight
- `@CacheEvict`: 999.0 after ~1 s, key disappears then reappears
- `@CachePut`: 999.0 in ~0 ms

Failure modes

- **Stale data:**
  12:00 cache 1200
  12:01 DB 999
  12:02 user reads 1200 → how stale may we be?

- **Stampede:**
  a hot key expires → 1,000 misses → 1,000 DB calls → coordinated loading, jittered TTL, `refreshAfterWrite`

- **Unbounded cache:** `new ConcurrentHashMap<>()` + a key per request → `OutOfMemoryError`

- **Local inconsistency:**
  instance A 1200,
  instance B 999,
  DB 999 → consistency is a business requirement

Caffeine vs Redis

	Caffeine	Redis
Location	JVM memory	Remote process
Latency	in-process	network round trip
Shared between instances	No	Yes
Survives app restart	No	Yes
Serialization	No	Yes
Operational complexity	Low	Higher
Best for	Local hot data	Shared / distributed cache

Should we cache this? Which cache?

Ask first

- 1. Is it actually expensive?
- 2. Is the same data requested repeatedly?
- 3. How often does it change?
- 4. How stale may it be?
- 5. Acceptable failure mode?
- 6. How many instances?
- 7. Is invalidation practical?

Three takeaways

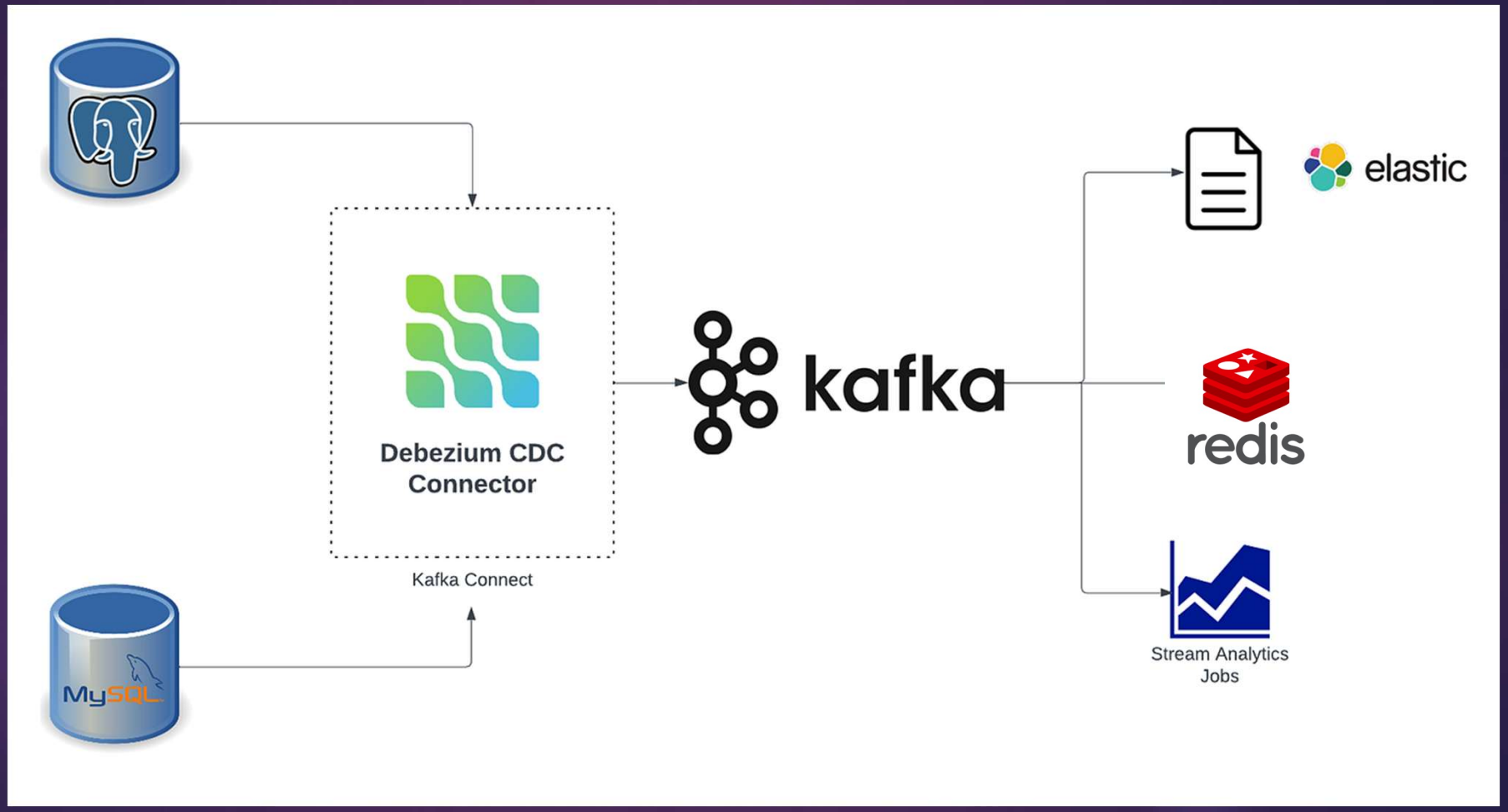
1. **A cache is not just a Map**
   It needs policies: memory, expiration, eviction, concurrency, observability

2. **Local and distributed caches solve different problems**
   Caffeine = fastest + local · Redis = shared + distributed

3. **@Cacheable is an abstraction**
   It moves the code, not the hard problems: invalidation, staleness, TTL, consistency, stampedes

The easiest part of caching is storing the value.
The engineering challenge is deciding when that value stops being correct.

Debezium


Debezium CDC
Connector
Kafka Connect
kafka
elastic
redis
Stream Analytics
Jobs
MySQL

QA Time!

